# Informe de Auditoría de Seguridad (SQLi y Seguridad Web)

## Portada

Repositorio auditado: adaldean/Perfumes
Commit auditado: 8dd1dc5b3d123149df7c32b4c1dbde297d7cc527
Fecha del informe: 22 de septiembre de 2026
Tipo de auditoría: Estática (revisión de código fuente)
Auditor: Copilot Coding Agent (en rol de auditor senior Django)

---

## 1. Alcance y limitaciones

### 1.1 Alcance

Se auditó exclusivamente el código actual del repositorio en el commit indicado, con foco en:

• SQL Injection (SQLi) y uso de consultas SQL.
• Autenticación/autorización y control de acceso.
• Aislamiento de recursos (IDOR en pagos/pedidos).
• Exposición de información por manejo de errores.
• Configuración de seguridad en despliegue.
• Riesgos de redirección abierta y CSRF/sesión.

Rutas y módulos principales revisados:
• myproject/urls.py
• apps/api/urls.py
• apps/core/api_urls.py
• apps/users/urls.py
• apps/api/views.py
• apps/api/auth_views.py
• apps/api/payments.py
• apps/catalog/api_views.py
• apps/catalog/views.py
• apps/orders/api_views.py
• apps/orders/payments.py
• myproject/settings.py
• render.yaml
• requirements.txt

### 1.2 Limitaciones

• Auditoría estática del commit: no se ejecutó la aplicación ni pruebas dinámicas de penetración.
• No se lanzaron ataques contra servicios externos.
• La evidencia corresponde al estado del código en el commit auditado.

---

## 2. Metodología

1. Mapeo de superficie de ataque (URLconf, vistas SSR y endpoints API/DRF).
2. Búsqueda de patrones SQL peligrosos (raw(), cursor.execute, extra, concatenación SQL).

3. Revisión de autenticación/autorización por endpoint y por método HTTP.
4. Revisión de flujos de pago/pedido para detectar IDOR y validación insuficiente.
5. Revisión de manejo de errores y configuración (DEBUG, SECRET_KEY, secretos en IaC).
6. Contraste con la guía docente de SQLi/seguridad web, adaptando respuestas al stack real (Django SSR + DRF).

---

## 3. Resumen ejecutivo

No se evidenció SQL Injection directa en el código inspeccionado: no se observaron raw(), cursor.execute ni extra() con SQL concatenado. Sin embargo, sí se identificaron vulnerabilidades críticas/altas en autenticación y control de acceso que permiten potencial toma de control administrativo y operaciones sobre recursos ajenos.

Riesgo global estimado: ALTO.

Hallazgos prioritarios:
• Endpoint público para creación de superusuario con credenciales hardcoded.
• Mutaciones de productos permitidas a cualquier usuario autenticado (no solo admin/staff).
• IDOR en creación/verificación de pagos por falta de amarre de pedido al usuario autenticado.
• Exposición de errores internos al cliente.
• Configuración insegura por defaults de DEBUG y SECRET_KEY.

---

## 4. Matriz de hallazgos

| ID | Hallazgo | Severidad | Impacto | Evidencia (archivo:línas) | Recomendación resumida |
|---|---|---|---|---|---|
| H-01 | Endpoint de creación de admin sin protección + credenciales codificadas | Crítica | Toma de control administrativo | apps/api/views.py:260-272, apps/api/urls.py:63, render.yaml:25-30 | Eliminar endpoint público o protegerlo con POST + IsAdminUser; mover bootstrap a comando de management sin credenciales por defecto |
| H-02 | Escritura de productos sin privilegio admin real | Alta | Alteración de catálogo por usuarios autenticados comunes | apps/api/views.py:65-79, apps/catalog/api_views.py:9-23 | Aplicar IsAdminUser (o permiso custom) para POST/PUT/PATCH/DELETE |

| ID | Hallazgo | Severidad | Impacto | Evidencia (archivo:línas) | Recomendación resumida |
|---|---|---|---|---|---|
| H-03 | IDOR en creación/verificación de pagos y pedido no ligado al usuario | Alta | Inicio/consulta de pago de pedidos ajenos | apps/api/payments.py:82, apps/orders/payments.py:83, apps/api/views.py:238-257, apps/orders/api_views.py:225-232 | Filtrar por Pedido.objects.get(id=pedido_id, usuario=request.user) salvo staff explícito |
| H-04 | Exposición de detalles internos por str(e) en respuestas | Media-Alta | Filtración de información técnica y de negocio | apps/api/views.py:175-179,256-257, apps/orders/api_views.py:71,152,189,232,266, apps/api/auth_views.py:121-124 | Mensajes genéricos al cliente + logging interno estructurado |
| H-05 | Configuración insegura por fallback de DEBUG y SECRET_KEY | Alta (en despliegue) | Debug activo y clave predecible en producción mal configurada | myproject/settings.py:10,12 | Hacer SECRET_KEY obligatoria; DEBUG=False por defecto fuera de local |
| H-06 | Posible open redirect en parámetro next | Media | Redirección a destino no confiable | apps/api/auth_views.py:166-167 | Validar con url_has_allowed_host_and_scheme y fallback seguro |
| H-07 | CSRF/métodos HTTP débiles en endpoints de sesión/consulta | Media | Riesgo CSRF o abuso de endpoints de consulta | apps/api/views.py:236-239, apps/api/auth_views.py:177-181, 595-613 | Exigir autenticación/autoría en verificar_pago; usar POST+CSRF para logout en sesión |
| H-08 | SQLi directa no evidenciada en código actual (riesgo residual de futuro) | Informativo | Riesgo latente si se introduce SQL raw no parametrizado | Búsqueda en repo: sin coincidencias de raw()/cursor.execute/extra() | Mantener ORM/parametrización; revisar PRs con política explícita |

---

## 5. Evidencia técnica detallada

### 5.1 SQL Injection (estado actual)

• No se encontraron usos de cursor.execute, QuerySet.raw() ni QuerySet.extra() en los módulos inspeccionados.
• Se observó uso de ORM de Django (filter, get, Q, get_or_404, serializers), por ejemplo:
• apps/catalog/views.py:10,27,40-45,50
• apps/api/views.py:83,88,93,132

Conclusión SQLi: no hay evidencia de SQLi directa en este commit. El riesgo principal actual está en acceso/autorización y configuración.

### 5.2 Hallazgo H-01: creación de admin sin protección

• Ruta expuesta: path('create-admin/', create_admin_user, ...) en apps/api/urls.py:63.
• Vista crea superusuario con credenciales hardcoded en apps/api/views.py:262-267.
• Misma contraseña débil en infraestructura: ADMIN_PASSWORD=securepassword en render.yaml:29-30.

Impacto: si el endpoint es accesible, un atacante podría forzar/probar creación o reutilización de cuenta admin predecible.

### 5.3 Hallazgo H-02: permisos de escritura de productos

• Comentario funcional declara “solo admin/staff”, pero implementación permite escritura a cualquier autenticado:
• apps/api/views.py:68 vs 74-79
• apps/catalog/api_views.py:12 vs 18-23

### 5.4 Hallazgo H-03: IDOR en pagos/pedidos

• Creación de payment intent usa Pedido.objects.get(id=pedido_id) sin filtrar por usuario:
• apps/api/payments.py:82
• apps/orders/payments.py:83
• Endpoint de creación acepta pedido_id autenticado:
• apps/api/views.py:162-170
• apps/orders/api_views.py:221-229
• Verificación de pago consultable por payment_intent_id arbitrario vía GET y sin autenticación explícita:
• apps/api/views.py:236-257

### 5.5 Hallazgo H-04: exposición de información

Se devuelven mensajes internos usando str(e) al cliente, p. ej.:
• apps/api/views.py:177,257
• apps/orders/api_views.py:71,152,189,232,266
• apps/api/auth_views.py:123

### 5.6 Hallazgo H-05: configuración insegura

• SECRET_KEY con fallback inseguro: myproject/settings.py:10
• DEBUG con fallback True: myproject/settings.py:12
• Aunque existen controles en if not DEBUG (SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, HSTS), su eficacia depende de desactivar debug (myproject/settings.py:113-127).

### 5.7 Hallazgo H-06: redirección abierta potencial

• next_url = request.GET.get('next', 'frontend:catalogo') y return redirect(next_url) en apps/api/auth_views.py:166-167 sin validación de host/esquema.

### 5.8 Hallazgo H-07: CSRF/sesión

• verificar_pago usa @csrf_exempt, método GET y sin autenticación en apps/api/views.py:236-239.
• logout_view admite GET (@require_http_methods(["GET", "POST"]) en login y logout_view sin restricción explícita POST en apps/api/auth_views.py:177-181), patrón menos robusto para sesión basada en cookie.
• stripe_webhook (apps/api/views.py:188-234) está exento de CSRF de forma justificada por verificación de firma Stripe.

---

## 6. Análisis SQL solicitado (didáctico)

### 6.1 Cómo rompe contexto ' OR 1=1--

Consulta vulnerable hipotética (no observada en este repo):

```
SELECT * FROM users
WHERE email = '<INPUT_USUARIO>' AND password = '<INPUT_PASSWORD>';
```

Si <INPUT_USUARIO> recibe ' OR 1=1--, podría transformarse en:

```
SELECT * FROM users
WHERE email = '' OR 1=1--' AND password = 'x';
```

Efecto:
• OR 1=1 vuelve la condición verdadera.
• -- comenta el resto de la línea en motores que lo aceptan (el comportamiento exacto depende del dialecto y si requiere espacio/terminador).

### 6.2 Contraste con este código Django

El código revisado usa ORM para filtros, lo que genera consultas parametrizadas internamente en los casos observados. Por tanto:
• No se recomienda usar blacklists de payloads.
• Sí se recomienda continuar con ORM y, si se requiere SQL raw, parametrizar explícitamente.

Ejemplo defensivo válido en Django:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT id, email FROM auth_user WHERE email = %s",
        [email],
    )
    rows = cursor.fetchall()
```

Preferencia: ORM (User.objects.filter(email=email)) salvo necesidad excepcional.

---

## 7. Remediaciones propuestas (con fragmentos de código)

### 7.1 Sustituir/privatizar creación de admin

```
# Opción recomendada: comando de management para bootstrap
# python manage.py createsuperuser
```

```
# Si se mantiene endpoint, mínimo:
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAdminUser

@api_view(["POST"])
@permission_classes([IsAdminUser])
def create_admin_user_private(request):
    # sin credenciales hardcoded; validar payload y registrar auditoría
    ...
```

### 7.2 Escritura de productos solo admin/staff

```
from rest_framework.permissions import AllowAny, IsAdminUser

def get_permissions(self):
    if self.request.method in ("GET", "HEAD", "OPTIONS"):
        return [AllowAny()]
    return [IsAdminUser()]
```

### 7.3 Aislamiento de pedidos en pagos (evitar IDOR)

```
# pasar request.user al servicio
pedido = Pedido.objects.get(id=pedido_id, usuario=request.user)
# o permitir staff explícito:
# if request.user.is_staff: Pedido.objects.get(id=pedido_id)
```

### 7.4 Eliminar str(e) en respuestas

```
logger.exception("Error creando pago")
return Response({"error": "No fue posible procesar la solicitud"}, status=400)
```

### 7.5 Endurecer verificar_pago

```
from django.contrib.auth.decorators import login_required
from django.views.decorators.http import require_GET

@login_required
@require_GET
def verificar_pago(request, payment_intent_id):
    # comprobar que el payment_intent_id pertenezca a un pago del usuario
    ...
```

### 7.6 Validar next

```
from django.utils.http import url_has_allowed_host_and_scheme

next_url = request.GET.get("next")
if next_url and url_has_allowed_host_and_scheme(next_url, allowed_hosts={request.get_host()}):
    return redirect(next_url)
return redirect("frontend:catalogo")
```

### 7.7 Configuración segura obligatoria

```
SECRET_KEY = os.environ["SECRET_KEY"]  # obligatorio
DEBUG = os.getenv("DEBUG", "False").lower() == "true"
if DEBUG and os.getenv("ENV") == "production":
    raise ImproperlyConfigured("DEBUG no permitido en producción")
```

---

## 8. Respuestas adaptadas a las preguntas de reflexión de la guía

> Contexto adaptado: este repositorio usa Django SSR + DRF, no una SPA Angular tipo Juice Shop.

1. -p 3000:3000 en Docker / cambio a -p 8080:3000
Mapea host:contenedor. En 8080:3000, se accede por http://localhost:8080 al servicio del contenedor en 3000.

2. Logs en tiempo real de contenedor
docker logs -f <contenedor>.

3. HTTP al cargar interfaz principal
Esperado: 200 OK en carga exitosa.

4. Ruta de login visible en arquitectura real
Según apps/users/urls.py, login SSR: /login/; JWT login API: /api/core/auth/login/ y /api/auth/login/ según URLconfs activas.

5. Función de -- en payload SQLi
Comenta el resto de la sentencia (según dialecto/formato), ayudando a neutralizar condiciones posteriores.

6. Por qué password puede ser arbitraria en bypass SQLi
Porque la inyección altera la cláusula WHERE para volverla verdadera y omitir validación real de contraseña.

7. Cuenta comprometida tras bypass clásico
Suele ser la primera fila que satisface la consulta (frecuentemente admin), pero depende de la consulta y orden de datos.

8. Pestaña para revisar JS en navegador
Network (filtro JS) y Sources para inspección de rutas/código cliente.

9. Dónde se localizaría una ruta oculta
En bundles/minificados JS del frontend (si existieran); en este proyecto, el routing principal relevante está en Django URLconf.

10. Confirmación visual de reto completado (conceptual laboratorio)
Indicador “SOLVED” o marca verde en scoreboard (aplicable a Juice Shop, no a este repo Django productivo).

11. Efecto de logout en storage/cookies
Se invalida sesión (cookie de sesión/token asociado), impidiendo reutilización normal del estado autenticado.

12. Cómo altera bender@... '-- la lógica
Cierra cadena y comenta validación posterior en consultas vulnerables concatenadas.

13. Indicador de suplantación de cuenta
Identidad mostrada en UI/token (username/email activo) cambia al usuario suplantado.

---

## 9. Plan de verificación posterior (post-remediación)

Comandos recomendados en CI/CD o entorno de revisión:

```
python manage.py check --deploy
python manage.py migrate --check
python manage.py test
```

Pruebas unitarias/pytest sugeridas:
1. create-admin inaccesible a no-admin / solo POST / sin credenciales hardcoded.
2. Escritura de productos rechazada (403) para usuario autenticado no staff.

3. Pago con pedido_id ajeno devuelve 404/403.
4. verificar_pago exige autenticación y pertenencia del pago al usuario.
5. Respuestas de error no exponen str(e) ni detalles internos.
6. next externo (https://evil.tld) es rechazado y redirige a ruta segura.
7. Caso SQLi conceptual en login Django:
• payload username: ' OR 1=1--
• authenticate() no autentica si no existe usuario real con credenciales válidas.

---

## 10. Conclusión

El repositorio no evidencia SQLi directa en el estado revisado, lo cual es consistente con el uso predominante del ORM de Django. No obstante, existen debilidades críticas/altas en administración, autorización e aislamiento de recursos que deben priorizarse de inmediato para evitar compromisos de cuenta privilegiada y acceso no autorizado a operaciones de pago.

---

## 11. Referencias de arquitectura/rutas auditadas

• myproject/urls.py
• apps/api/urls.py
• apps/core/api_urls.py
• apps/users/urls.py